All of Us TB x T2D

Bindu Edupulapati

Install Packages

DO THIS BEFORE ANYTHING ELSE! Otherwise, it will run into an error where R crashes at the end.

```
install.packages("xfun")
```

```
Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)
```

```
install.packages(c("gt", "knitr", "rmarkdown", "htmltools", "cli", "rlang"))
```

```
Installing packages into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)
```

```
library(gtsummary)
```

Code to Extract Data Set

The following code will create 4 different datasets based on the 3 cohorts made and the demographics information selected.

```
# Copying code output from All of Us to generate data sets

library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0
──
✓ dplyr     1.2.0     ✓ readr     2.1.5
✓ forcats   1.0.0     ✓ stringr   1.5.1
✓ ggplot2   3.5.1     ✓ tibble    3.2.1
✓ lubridate 1.9.3     ✓ tidyr     1.3.1
✓ purrr     1.0.2
── Conflicts ────────────────────────────────────────── tidyverse_conflicts()
──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
library(bigrquery)
```

```
# This query represents data set "Draft #3" for domain "person" and was
generated for All of Us Controlled Tier Data set v8
dataset_14569019_person_sql <- paste("
    SELECT
        person.person_id,
        person.gender_concept_id,
        p_gender_concept.concept_name as gender,
        person.birth_datetime as date_of_birth,
        person.race_concept_id,
        p_race_concept.concept_name as race,
        person.ethnicity_concept_id,
        p_ethnicity_concept.concept_name as ethnicity,
        person.sex_at_birth_concept_id,
        p_sex_at_birth_concept.concept_name as sex_at_birth,
        person.self_reported_category_concept_id,
        p_self_reported_category_concept.concept_name as
self_reported_category
    FROM
        `person` person
    LEFT JOIN
        `concept` p_gender_concept
            ON person.gender_concept_id = p_gender_concept.concept_id
    LEFT JOIN
        `concept` p_race_concept
            ON person.race_concept_id = p_race_concept.concept_id
    LEFT JOIN
        `concept` p_ethnicity_concept
            ON person.ethnicity_concept_id = p_ethnicity_concept.concept_id
    LEFT JOIN
        `concept` p_sex_at_birth_concept
            ON person.sex_at_birth_concept_id =
p_sex_at_birth_concept.concept_id
    LEFT JOIN
        `concept` p_self_reported_category_concept
            ON person.self_reported_category_concept_id =
p_self_reported_category_concept.concept_id
    WHERE
        person.PERSON_ID IN (SELECT
            distinct person_id
        FROM
            `cb_search_person` cb_search_person
        WHERE
            cb_search_person.person_id IN (SELECT
                criteria.person_id
            FROM
                (SELECT
                    DISTINCT person_id, entry_date, concept_id
                FROM
                    `cb_search_all_events`
                WHERE
```

```
                    (concept_id IN(SELECT
                        DISTINCT c.concept_id
                    FROM
                        `cb_criteria` c
                    JOIN
                        (SELECT
                            CAST(cr.id as string) AS id
                        FROM
                            `cb_criteria` cr
                        WHERE
                            concept_id IN (44832194, 44828795, 44832193,
44836915, 44826461, 44833366, 1567956, 44829879, 44829882, 44824073,
44824072, 44831045, 44831047, 44836914, 44833367, 44827616, 44827617,
44836916, 44829878, 44826460, 44819500)
                            AND full_text LIKE '%_rank1]%'      ) a
                            ON (c.path LIKE CONCAT('%.', a.id, '.%')
                            OR c.path LIKE CONCAT('%.', a.id)
                            OR c.path LIKE CONCAT(a.id, '.%')
                            OR c.path = a.id)
                    WHERE
                        is_standard = 0
                        AND is_selectable = 1)
                    AND is_standard = 0 )) criteria )
              AND cb_search_person.person_id IN (SELECT
                  criteria.person_id
              FROM
                  (SELECT
                      DISTINCT person_id, entry_date, concept_id
                  FROM
                      `cb_search_all_events`
                  WHERE
                      (concept_id IN(SELECT
                          DISTINCT c.concept_id
                      FROM
                          `cb_criteria` c
                      JOIN
                          (SELECT
                              CAST(cr.id as string) AS id
                          FROM
                              `cb_criteria` cr
                          WHERE
                              concept_id IN (1567250, 44819327, 44835579,
44820533, 44825121, 1567249, 44819316, 1567252, 44828631, 44820542, 1567258,
44826304)
                              AND full_text LIKE '%_rank1]%'      ) a
                              ON (c.path LIKE CONCAT('%.', a.id, '.%')
                              OR c.path LIKE CONCAT('%.', a.id)
                              OR c.path LIKE CONCAT(a.id, '.%')
                              OR c.path = a.id)
                      WHERE
```

```
                        is_standard = 0
                        AND is_selectable = 1)
                    AND is_standard = 0 )) criteria ) )", sep="")

# Formulate a Cloud Storage destination path for the data exported from
BigQuery.
# NOTE: By default data exported multiple times on the same day will
overwrite older copies.
#       But data exported on a different days will write to a new location so
that historical
#       copies can be kept as the dataset definition is changed.
person_14569019_path <- file.path(
  Sys.getenv("WORKSPACE_BUCKET"),
  "bq_exports",
  Sys.getenv("OWNER_EMAIL"),
  strftime(lubridate::now(), "%Y%m%d"),  # Comment out this line if you want
the export to always overwrite.
  "person_14569019",
  "person_14569019_*.csv")
message(str_glue('The data will be written to {person_14569019_path}. Use
this path when reading ',
                 'the data into your notebooks in the future.'))
```

```
The data will be written to gs://fc-secure-292a5554-4759-41e1-8212-
3e78cd99c920/bq_exports/bedupul@researchallofus.org/20260302/person_14569019/
person_14569019_*.csv. Use this path when reading the data into your
notebooks in the future.
```

```
# Perform the query and export the dataset to Cloud Storage as CSV files.
# NOTE: You only need to run `bq_table_save` once. After that, you can
#       just read data from the CSVs in Cloud Storage.
bq_table_save(
  bq_dataset_query(Sys.getenv("WORKSPACE_CDR"), dataset_14569019_person_sql,
billing = Sys.getenv("GOOGLE_PROJECT")),
  person_14569019_path,
  destination_format = "CSV")



# Read the data directly from Cloud Storage into memory.
# NOTE: Alternatively you can `gsutil -m cp {person_14569019_path}` to copy
these files
#       to the Jupyter disk.
read_bq_export_from_workspace_bucket <- function(export_path) {
  col_types <- cols(gender = col_character(), race = col_character(),
ethnicity = col_character(), sex_at_birth = col_character(),
self_reported_category = col_character())
  bind_rows(
    map(system2('gsutil', args = c('ls', export_path), stdout = TRUE, stderr
= TRUE),
        function(csv) {
```

```
          message(str_glue('Loading {csv}.'))
          chunk <- read_csv(pipe(str_glue('gsutil cat {csv}')), col_types =
col_types, show_col_types = FALSE)
          if (is.null(col_types)) {
            col_types <- spec(chunk)
          }
          chunk
        }))
}
dataset_14569019_person_df <-
read_bq_export_from_workspace_bucket(person_14569019_path)

Loading gs://fc-secure-292a5554-4759-41e1-8212-
3e78cd99c920/bq_exports/bedupul@researchallofus.org/20260302/person_14569019/
person_14569019_000000000000.csv.

dim(dataset_14569019_person_df)

[1] 616  12

head(dataset_14569019_person_df, 5)

# A tibble: 5 × 12
  person_id gender_concept_id gender    date_of_birth      race_concept_id
race
      <dbl>             <dbl> <chr>     <chr>                        <dbl>
<chr>
1   1660274            903096 PMI: Skip 1956-06-15 00:00:…        45882607
None…
2   2005168          45878463 Female    1965-06-15 00:00:…        45882607
None…
3   1337651          45878463 Female    1937-06-15 00:00:…        45882607
None…
4   1123306          45878463 Female    1947-06-15 00:00:…        45882607
None…
5   1926132          45880669 Male      1993-06-15 00:00:…        45882607
None…
# ℹ 6 more variables: ethnicity_concept_id <dbl>, ethnicity <chr>,
#   sex_at_birth_concept_id <dbl>, sex_at_birth <chr>,
#   self_reported_category_concept_id <dbl>, self_reported_category <chr>

library(tidyverse)
library(bigrquery)

# This query represents dataset "Draft #3" for domain "condition" and was
generated for All of Us Controlled Tier Dataset v8
dataset_14569019_condition_sql <- paste("
    SELECT
        c_occurrence.person_id,
        c_occurrence.condition_concept_id,
        c_standard_concept.concept_name as standard_concept_name,
```

```
        c_occurrence.condition_start_datetime,
        c_occurrence.condition_end_datetime,
        visit.concept_name as visit_occurrence_concept_name
    FROM
        ( SELECT
            *
        FROM
            `condition_occurrence` c_occurrence
        WHERE
            (
                condition_source_concept_id IN (SELECT
                    DISTINCT c.concept_id
                FROM
                    `cb_criteria` c
                JOIN
                    (SELECT
                        CAST(cr.id as string) AS id
                    FROM
                        `cb_criteria` cr
                    WHERE
                        concept_id IN (1326492, 1326493, 1567257, 1567956,
1567958, 1567959, 1567960, 1567964, 1567965, 1567966, 1567969, 1567971,
35205451, 35205452, 35205453, 35205454, 35205455, 35205456, 35205457,
35205458, 35205460, 35205461, 35205462, 35205466, 35205467, 35205468,
35205469, 35206881, 35206882, 37200198, 37200199, 37200200, 37200201,
37200202, 37200203, 37200204, 37200205, 37200206, 37200207, 37200208,
37200209, 37200210, 37200211, 37200212, 37200213, 37200214, 37200215,
37200216, 37200217, 37200218, 37200219, 37200220, 37200221, 37200222,
37200223, 37200224, 37200225, 37200227, 37200228, 37200229, 37200230,
37200232, 37200233, 37200234, 37200235, 37200237, 37200238, 37200239,
37200240, 37200242, 37200243, 37200244, 37200245, 37200246, 37200247,
37200248, 37200249, 37200251, 37200252, 37200253, 37200254, 44819312,
44819316, 44819318, 44819323, 44819327, 44819328, 44819333, 44819339,
44819340, 44819341, 44819346, 44820524, 44820530, 44820531, 44820533,
44820536,
 44820543, 44821635, 44821636, 44821639, 44821640, 44821642, 44821643,
44821644, 44821645, 44821647, 44821651, 44822780, 44822782, 44822783,
44822784, 44822785, 44822786, 44822791, 44822793, 44822796, 44822798,
44822807, 44823934, 44823938, 44823942, 44823943, 44823948, 44825100,
44825102, 44825111, 44825121, 44825122, 44825123, 44825131, 44826294,
44826295, 44826303, 44826319, 44826320, 44827453, 44827459, 44827460,
44827461, 44827471, 44827474, 44828631, 44828632, 44828633, 44828635,
44828637, 44828640, 44828648, 44828654, 44828655, 44828659, 44828660,
44829704, 44829706, 44829710, 44829713, 44829725, 44829727, 44830888,
44830889, 44830891, 44830892, 44830895, 44830896, 44830899, 44830904,
44830905, 44830907, 44832040, 44832043, 44832044, 44832045, 44832061,
44833187, 44833188, 44833191, 44833203, 44833208, 44833216, 44834391,
44834395, 44834396, 44834398, 44834405, 44834406, 44834407, 44834411,
44834413, 44835573, 44835574, 44835580, 44835587, 44835588, 44835590,
44835594, 44835595,
```

```
 44835598, 44836743, 44836747, 44836750, 44836753, 44836755, 44836756,
44836758, 44836762, 44836764, 44836914, 45533019, 45533020, 45533021,
45533022, 45533023, 45537961, 45537962, 45542517, 45542518, 45542738,
45547394, 45547396, 45547625, 45547626, 45547627, 45552158, 45552159,
45552160, 45552385, 45552386, 45557112, 45557113, 45561686, 45561949,
45566731, 45571417, 45571418, 45576443, 45581104, 45581352, 45581353,
45581354, 45581355, 45585901, 45585903, 45586139, 45586140, 45591027,
45591029, 45591030, 45591031, 45595576, 45595577, 45595578, 45595579,
45595797, 45595798, 45595799, 45600400, 45600641, 45600642, 45605193,
45605195, 45605401, 45605402, 45605403, 45605404, 45605405)
                        AND full_text LIKE '%_rank1]%'      ) a
                        ON (c.path LIKE CONCAT('%.', a.id, '.%')
                        OR c.path LIKE CONCAT('%.', a.id)
                        OR c.path LIKE CONCAT(a.id, '.%')
                        OR c.path = a.id)
                WHERE
                    is_standard = 0
                    AND is_selectable = 1)
            )
            AND (
                c_occurrence.PERSON_ID IN (SELECT
                    distinct person_id
                FROM
                    `cb_search_person` cb_search_person
                WHERE
                    cb_search_person.person_id IN (SELECT
                        criteria.person_id
                    FROM
                        (SELECT
                            DISTINCT person_id, entry_date, concept_id
                        FROM
                            `cb_search_all_events`
                        WHERE
                            (concept_id IN(SELECT
                                DISTINCT c.concept_id
                            FROM
                                `cb_criteria` c
                            JOIN
                                (SELECT
                                    CAST(cr.id as string) AS id
                                FROM
                                    `cb_criteria` cr
                                WHERE
                                    concept_id IN (44832194, 44828795,
44832193, 44836915, 44826461, 44833366, 1567956, 44829879, 44829882,
44824073, 44824072, 44831045, 44831047, 44836914, 44833367, 44827616,
44827617, 44836916, 44829878, 44826460, 44819500)
                                    AND full_text LIKE '%_rank1]%'      ) a
                                    ON (c.path LIKE CONCAT('%.', a.id, '.%')
                                    OR c.path LIKE CONCAT('%.', a.id)
```

```
                                    OR c.path LIKE CONCAT(a.id, '.%')
                                    OR c.path = a.id)
                                WHERE
                                    is_standard = 0
                                    AND is_selectable = 1)
                                AND is_standard = 0 )) criteria )
                    AND cb_search_person.person_id IN (SELECT
                        criteria.person_id
                    FROM
                        (SELECT
                            DISTINCT person_id, entry_date, concept_id
                        FROM
                            `cb_search_all_events`
                        WHERE
                            (concept_id IN(SELECT
                                DISTINCT c.concept_id
                            FROM
                                `cb_criteria` c
                            JOIN
                                (SELECT
                                    CAST(cr.id as string) AS id
                                FROM
                                    `cb_criteria` cr
                                WHERE
                                    concept_id IN (1567250, 44819327,
44835579, 44820533, 44825121, 1567249, 44819316, 1567252, 44828631, 44820542,
1567258, 44826304)
                                    AND full_text LIKE '%_rank1]%'      ) a
                                    ON (c.path LIKE CONCAT('%.', a.id, '.%')
                                    OR c.path LIKE CONCAT('%.', a.id)
                                    OR c.path LIKE CONCAT(a.id, '.%')
                                    OR c.path = a.id)
                                WHERE
                                    is_standard = 0
                                    AND is_selectable = 1)
                                AND is_standard = 0 )) criteria ) )
            )) c_occurrence
    LEFT JOIN
        `concept` c_standard_concept
            ON c_occurrence.condition_concept_id =
c_standard_concept.concept_id
    LEFT JOIN
        `visit_occurrence` v
            ON c_occurrence.visit_occurrence_id = v.visit_occurrence_id
    LEFT JOIN
        `concept` visit
            ON v.visit_concept_id = visit.concept_id", sep="")

# Formulate a Cloud Storage destination path for the data exported from
BigQuery.
```

```
# NOTE: By default data exported multiple times on the same day will
overwrite older copies.
#       But data exported on a different days will write to a new location so
that historical
#       copies can be kept as the dataset definition is changed.
condition_14569019_path <- file.path(
  Sys.getenv("WORKSPACE_BUCKET"),
  "bq_exports",
  Sys.getenv("OWNER_EMAIL"),
  strftime(lubridate::now(), "%Y%m%d"),  # Comment out this line if you want
the export to always overwrite.
  "condition_14569019",
  "condition_14569019_*.csv")
message(str_glue('The data will be written to {condition_14569019_path}. Use
this path when reading ',
                 'the data into your notebooks in the future.'))
```

```
The data will be written to gs://fc-secure-292a5554-4759-41e1-8212-
3e78cd99c920/bq_exports/bedupul@researchallofus.org/20260302/condition_145690
19/condition_14569019_*.csv. Use this path when reading the data into your
notebooks in the future.
```

```
# Perform the query and export the dataset to Cloud Storage as CSV files.
# NOTE: You only need to run `bq_table_save` once. After that, you can
#       just read data from the CSVs in Cloud Storage.
bq_table_save(
  bq_dataset_query(Sys.getenv("WORKSPACE_CDR"),
dataset_14569019_condition_sql, billing = Sys.getenv("GOOGLE_PROJECT")),
  condition_14569019_path,
  destination_format = "CSV")
```

```
# Read the data directly from Cloud Storage into memory.
# NOTE: Alternatively you can `gsutil -m cp {condition_14569019_path}` to
copy these files
#       to the Jupyter disk.
read_bq_export_from_workspace_bucket <- function(export_path) {
  col_types <- cols(standard_concept_name = col_character(),
visit_occurrence_concept_name = col_character())
  bind_rows(
    map(system2('gsutil', args = c('ls', export_path), stdout = TRUE, stderr
= TRUE),
        function(csv) {
          message(str_glue('Loading {csv}.'))
          chunk <- read_csv(pipe(str_glue('gsutil cat {csv}')), col_types =
col_types, show_col_types = FALSE)
          if (is.null(col_types)) {
            col_types <- spec(chunk)
          }
          chunk
```

```
        }))
}
dataset_14569019_condition_df <-
read_bq_export_from_workspace_bucket(condition_14569019_path)
```

```
Loading gs://fc-secure-292a5554-4759-41e1-8212-
3e78cd99c920/bq_exports/bedupul@researchallofus.org/20260302/condition_145690
19/condition_14569019_000000000000.csv.
```

```
dim(dataset_14569019_condition_df)
```

```
[1] 60004     6
```

```
head(dataset_14569019_condition_df, 5)
```

```
# A tibble: 5 × 6
  person_id condition_concept_id standard_concept_name
condition_start_date…¹
      <dbl>                <dbl> <chr>                     <chr>
1   2772212               253954 Pulmonary tuberculosis   1995-07-14 00:00:00
U…
2   3501656             43531616 Dermopathy due to type … 2018-12-26 06:39:21
U…
3   1786780               376065 Disorder of nervous sys… 2016-06-14 00:00:00
U…
4   1039573              4008576 Diabetes mellitus witho… 2015-06-30 06:19:48
U…
5   2138263               376065 Disorder of nervous sys… 2021-07-21 00:00:00
U…
# ℹ abbreviated name: ¹condition_start_datetime
# ℹ 2 more variables: condition_end_datetime <chr>,
#   visit_occurrence_concept_name <chr>
```

```
library(tidyverse)
library(bigrquery)

# This query represents dataset "Draft #3" for domain "measurement" and was
generated for All of Us Controlled Tier Dataset v8
dataset_14569019_measurement_sql <- paste("
    SELECT
        measurement.person_id,
        measurement.measurement_concept_id,
        m_standard_concept.concept_name as standard_concept_name,
        measurement.measurement_datetime,
        measurement.value_as_number,
        measurement.range_low,
        measurement.range_high,
        measurement.unit_source_value
    FROM
        ( SELECT
            *
```

```
        FROM
            `measurement` measurement
        WHERE
            (
                measurement_concept_id IN (SELECT
                    DISTINCT c.concept_id
                FROM
                    `cb_criteria` c
                JOIN
                    (SELECT
                        CAST(cr.id as string) AS id
                    FROM
                        `cb_criteria` cr
                    WHERE
                        concept_id IN (3004410, 3009413, 3011873, 3017250,
3037110, 3038553, 4012477, 4102154, 4229586)
                        AND full_text LIKE '%_rank1]%'      ) a
                        ON (c.path LIKE CONCAT('%.', a.id, '.%')
                        OR c.path LIKE CONCAT('%.', a.id)
                        OR c.path LIKE CONCAT(a.id, '.%')
                        OR c.path = a.id)
                WHERE
                    is_standard = 1
                    AND is_selectable = 1)
            )
            AND (
                measurement.PERSON_ID IN (SELECT
                    distinct person_id
                FROM
                    `cb_search_person` cb_search_person
                WHERE
                    cb_search_person.person_id IN (SELECT
                        criteria.person_id
                    FROM
                        (SELECT
                            DISTINCT person_id, entry_date, concept_id
                        FROM
                            `cb_search_all_events`
                        WHERE
                            (concept_id IN(SELECT
                                DISTINCT c.concept_id
                            FROM
                                `cb_criteria` c
                            JOIN
                                (SELECT
                                    CAST(cr.id as string) AS id
                                FROM
                                    `cb_criteria` cr
                                WHERE
                                    concept_id IN (44832194, 44828795,
```

```
44832193, 44836915, 44826461, 44833366, 1567956, 44829879, 44829882,
44824073, 44824072, 44831045, 44831047, 44836914, 44833367, 44827616,
44827617, 44836916, 44829878, 44826460, 44819500)
                                    AND full_text LIKE '%_rank1]%'      ) a
                                    ON (c.path LIKE CONCAT('%.', a.id, '.%')
                                    OR c.path LIKE CONCAT('%.', a.id)
                                    OR c.path LIKE CONCAT(a.id, '.%')
                                    OR c.path = a.id)
                                WHERE
                                    is_standard = 0
                                    AND is_selectable = 1)
                                AND is_standard = 0 )) criteria )
                        AND cb_search_person.person_id IN (SELECT
                            criteria.person_id
                        FROM
                            (SELECT
                                DISTINCT person_id, entry_date, concept_id
                            FROM
                                `cb_search_all_events`
                            WHERE
                                (concept_id IN(SELECT
                                    DISTINCT c.concept_id
                                FROM
                                    `cb_criteria` c
                                JOIN
                                    (SELECT
                                        CAST(cr.id as string) AS id
                                    FROM
                                        `cb_criteria` cr
                                    WHERE
                                        concept_id IN (1567250, 44819327,
44835579, 44820533, 44825121, 1567249, 44819316, 1567252, 44828631, 44820542,
1567258, 44826304)
                                        AND full_text LIKE '%_rank1]%'      ) a
                                        ON (c.path LIKE CONCAT('%.', a.id, '.%')
                                        OR c.path LIKE CONCAT('%.', a.id)
                                        OR c.path LIKE CONCAT(a.id, '.%')
                                        OR c.path = a.id)
                                WHERE
                                    is_standard = 0
                                    AND is_selectable = 1)
                                AND is_standard = 0 )) criteria ) )
            )) measurement
    LEFT JOIN
        `concept` m_standard_concept
            ON measurement.measurement_concept_id =
m_standard_concept.concept_id", sep="")

# Formulate a Cloud Storage destination path for the data exported from
BigQuery.
```

```
# NOTE: By default data exported multiple times on the same day will
overwrite older copies.
#       But data exported on a different days will write to a new location so
that historical
#       copies can be kept as the dataset definition is changed.
measurement_14569019_path <- file.path(
  Sys.getenv("WORKSPACE_BUCKET"),
  "bq_exports",
  Sys.getenv("OWNER_EMAIL"),
  strftime(lubridate::now(), "%Y%m%d"),  # Comment out this line if you want
the export to always overwrite.
  "measurement_14569019",
  "measurement_14569019_*.csv")
message(str_glue('The data will be written to {measurement_14569019_path}.
Use this path when reading ',
                 'the data into your notebooks in the future.'))
```

```
The data will be written to gs://fc-secure-292a5554-4759-41e1-8212-
3e78cd99c920/bq_exports/bedupul@researchallofus.org/20260302/measurement_1456
9019/measurement_14569019_*.csv. Use this path when reading the data into
your notebooks in the future.
```

```
# Perform the query and export the dataset to Cloud Storage as CSV files.
# NOTE: You only need to run `bq_table_save` once. After that, you can
#       just read data from the CSVs in Cloud Storage.
bq_table_save(
  bq_dataset_query(Sys.getenv("WORKSPACE_CDR"),
dataset_14569019_measurement_sql, billing = Sys.getenv("GOOGLE_PROJECT")),
  measurement_14569019_path,
  destination_format = "CSV")
```

```
# Read the data directly from Cloud Storage into memory.
# NOTE: Alternatively you can `gsutil -m cp {measurement_14569019_path}` to
copy these files
#       to the Jupyter disk.
read_bq_export_from_workspace_bucket <- function(export_path) {
  col_types <- cols(standard_concept_name = col_character(),
unit_source_value = col_character())
  bind_rows(
    map(system2('gsutil', args = c('ls', export_path), stdout = TRUE, stderr
= TRUE),
        function(csv) {
          message(str_glue('Loading {csv}.'))
          chunk <- read_csv(pipe(str_glue('gsutil cat {csv}')), col_types =
col_types, show_col_types = FALSE)
          if (is.null(col_types)) {
            col_types <- spec(chunk)
          }
          chunk
```

```
        }))
}
dataset_14569019_measurement_df <-
read_bq_export_from_workspace_bucket(measurement_14569019_path)

Loading gs://fc-secure-292a5554-4759-41e1-8212-
3e78cd99c920/bq_exports/bedupul@researchallofus.org/20260302/measurement_1456
9019/measurement_14569019_000000000000.csv.

dim(dataset_14569019_measurement_df)

[1] 51245     8

head(dataset_14569019_measurement_df, 5)

# A tibble: 5 × 8
  person_id measurement_concept_id standard_concept_name
measurement_datetime
      <dbl>                  <dbl> <chr>                  <chr>
1   4652009                4012477 Glucose tolerance test 2006-03-17 07:31:00
U…
2   1178344                4012477 Glucose tolerance test 2014-11-21 15:07:00
U…
3   5748226                4012477 Glucose tolerance test 2015-10-30 07:29:00
U…
4   5748226                4012477 Glucose tolerance test 2002-11-12 09:15:00
U…
5   1178344                4012477 Glucose tolerance test 2014-11-28 12:59:00
U…
# ℹ 4 more variables: value_as_number <dbl>, range_low <dbl>, range_high
<dbl>,
#   unit_source_value <chr>

library(tidyverse)
library(bigrquery)

# This query represents dataset "Draft #3" for domain "survey" and was
generated for All of Us Controlled Tier Dataset v8
dataset_14569019_survey_sql <- paste("
    SELECT
        answer.person_id,
        answer.question,
        answer.answer_concept_id,
        answer.answer
    FROM
        `ds_survey` answer
    WHERE
        (
            question_concept_id IN (1585940, 836800, 836848)
        )
        AND (
```

```
            answer.PERSON_ID IN (SELECT
                distinct person_id
            FROM
                `cb_search_person` cb_search_person
            WHERE
                cb_search_person.person_id IN (SELECT
                    criteria.person_id
                FROM
                    (SELECT
                        DISTINCT person_id, entry_date, concept_id
                    FROM
                        `cb_search_all_events`
                    WHERE
                        (concept_id IN(SELECT
                            DISTINCT c.concept_id
                        FROM
                            `cb_criteria` c
                        JOIN
                            (SELECT
                                CAST(cr.id as string) AS id
                            FROM
                                `cb_criteria` cr
                            WHERE
                                concept_id IN (44832194, 44828795, 44832193,
44836915, 44826461, 44833366, 1567956, 44829879, 44829882, 44824073,
44824072, 44831045, 44831047, 44836914, 44833367, 44827616, 44827617,
44836916, 44829878, 44826460, 44819500)
                                AND full_text LIKE '%_rank1]%'      ) a
                                ON (c.path LIKE CONCAT('%.', a.id, '.%')
                                OR c.path LIKE CONCAT('%.', a.id)
                                OR c.path LIKE CONCAT(a.id, '.%')
                                OR c.path = a.id)
                        WHERE
                            is_standard = 0
                            AND is_selectable = 1)
                        AND is_standard = 0 )) criteria )
                AND cb_search_person.person_id IN (SELECT
                    criteria.person_id
                FROM
                    (SELECT
                        DISTINCT person_id, entry_date, concept_id
                    FROM
                        `cb_search_all_events`
                    WHERE
                        (concept_id IN(SELECT
                            DISTINCT c.concept_id
                        FROM
                            `cb_criteria` c
                        JOIN
                            (SELECT
```

```
                                CAST(cr.id as string) AS id
                            FROM
                                `cb_criteria` cr
                            WHERE
                                concept_id IN (1567250, 44819327, 44835579,
44820533, 44825121, 1567249, 44819316, 1567252, 44828631, 44820542, 1567258,
44826304)
                                AND full_text LIKE '%_rank1]%'      ) a
                                ON (c.path LIKE CONCAT('%.', a.id, '.%')
                                OR c.path LIKE CONCAT('%.', a.id)
                                OR c.path LIKE CONCAT(a.id, '.%')
                                OR c.path = a.id)
                        WHERE
                            is_standard = 0
                            AND is_selectable = 1)
                        AND is_standard = 0 )) criteria ) )
        )", sep="")

# Formulate a Cloud Storage destination path for the data exported from
BigQuery.
# NOTE: By default data exported multiple times on the same day will
overwrite older copies.
#       But data exported on a different days will write to a new location so
that historical
#       copies can be kept as the dataset definition is changed.
survey_14569019_path <- file.path(
  Sys.getenv("WORKSPACE_BUCKET"),
  "bq_exports",
  Sys.getenv("OWNER_EMAIL"),
  strftime(lubridate::now(), "%Y%m%d"),  # Comment out this line if you want
the export to always overwrite.
  "survey_14569019",
  "survey_14569019_*.csv")
message(str_glue('The data will be written to {survey_14569019_path}. Use
this path when reading ',
                 'the data into your notebooks in the future.'))
```

```
The data will be written to gs://fc-secure-292a5554-4759-41e1-8212-
3e78cd99c920/bq_exports/bedupul@researchallofus.org/20260302/survey_14569019/
survey_14569019_*.csv. Use this path when reading the data into your
notebooks in the future.
```

```
# Perform the query and export the dataset to Cloud Storage as CSV files.
# NOTE: You only need to run `bq_table_save` once. After that, you can
#       just read data from the CSVs in Cloud Storage.
bq_table_save(
  bq_dataset_query(Sys.getenv("WORKSPACE_CDR"), dataset_14569019_survey_sql,
billing = Sys.getenv("GOOGLE_PROJECT")),
  survey_14569019_path,
  destination_format = "CSV")
```

```r
# Read the data directly from Cloud Storage into memory.
# NOTE: Alternatively you can `gsutil -m cp {survey_14569019_path}` to copy
these files
#       to the Jupyter disk.
read_bq_export_from_workspace_bucket <- function(export_path) {
  col_types <- cols(survey = col_character(), question = col_character(),
answer = col_character())
  bind_rows(
    map(system2('gsutil', args = c('ls', export_path), stdout = TRUE, stderr
= TRUE),
        function(csv) {
          message(str_glue('Loading {csv}.'))
          chunk <- read_csv(pipe(str_glue('gsutil cat {csv}')), col_types =
col_types, show_col_types = FALSE)
          if (is.null(col_types)) {
            col_types <- spec(chunk)
          }
          chunk
        }))
}
dataset_14569019_survey_df <-
read_bq_export_from_workspace_bucket(survey_14569019_path)
```

```
Loading gs://fc-secure-292a5554-4759-41e1-8212-
3e78cd99c920/bq_exports/bedupul@researchallofus.org/20260302/survey_14569019/
survey_14569019_000000000000.csv.

Warning: The following named parsers don't match the column names: survey
```

```r
dim(dataset_14569019_survey_df)
```

```
[1] 996   4
```

```r
head(dataset_14569019_survey_df, 5)
```

```
# A tibble: 5 × 4
  person_id question                                      answer_concept_id
answer
      <dbl> <chr>                                                     <dbl>
<chr>
1   2003952 Including yourself, who in your family has…          43529932
Inclu…
2   4959085 Including yourself, who in your family has…          43529668
Inclu…
3   1371218 Including yourself, who in your family has…            903096
PMI: …
4   1876160 Including yourself, who in your family has…            903096
PMI: …
```

```
5   2376164 Including yourself, who in your family has…          43529937
Inclu…
```

Identifying Unique Variables to Create Code Book

Each variable in All Of Us has an associated ID with its name (e.g., condition name or demographic). The following code will identify the IDs. This is done for all the variables except for visit_occurrence, as there were many different codes associated with the same value. The codes generated from this are on a seperate Excel.

```
# identifying id associated with each unique value for each variable
# conditions data set
unique(dataset_14569019_condition_df[, c("condition_concept_id",
"standard_concept_name")]) # diagnosis
```

```
# A tibble: 91 × 2
   condition_concept_id standard_concept_name
                  <dbl> <chr>
 1               253954 Pulmonary tuberculosis
 2             43531616 Dermopathy due to type 2 diabetes mellitus
 3               376065 Disorder of nervous system due to type 2 diabetes
melli…
 4              4008576 Diabetes mellitus without complication
 5             43531578 Chronic kidney disease due to type 2 diabetes
mellitus
 6             37016349 Hyperglycemia due to type 2 diabetes mellitus
 7               201826 Type 2 diabetes mellitus
 8               192818 Tuberculosis of male genital organs
 9              4193704 Type 2 diabetes mellitus without complication
10               442793 Complication due to diabetes mellitus
# ℹ 81 more rows
```

```
unique(dataset_14569019_condition_df$visit_occurrence_concept_name) #
location of diagnosis
```

```
 [1] NA
 [2] "Ambulatory Rehabilitation Visit"
 [3] "Ambulatory Mammography Clinic / Center"
 [4] "Unknown Value (but present in data)"
 [5] "Pharmacy visit"
 [6] "Home Visit"
 [7] "Office Visit"
 [8] "Telehealth"
 [9] "Inpatient Psychiatric Facility"
[10] "Nursing Facility"
[11] "Pharmacy"
[12] "Emergency Room and Inpatient Visit"
[13] "Ambulatory Endoscopy Cinic / Center"
[14] "Ambulatory Surgical Center"
```

```
[15] "Outpatient Hospital"
[16] "Outpatient Visit"
[17] "Emergency Room Visit"
[18] "Ambulatory Infusion Therapy Clinic / Center"
[19] "Observation Room"
[20] "Ambulatory Clinic / Center"
[21] "Ambulatory Podiatric Clinic / Center"
[22] "Comprehensive Inpatient Rehabilitation Facility"
[23] "Inpatient Visit"
[24] "Laboratory Visit"
[25] "Non-hospital institution Visit"
[26] "Ambulatory Radiology Clinic / Center"
```

```
# measurement data set
unique(dataset_14569019_measurement_df[, c("measurement_concept_id",
"standard_concept_name")]) # lab test name
```

```
# A tibble: 7 × 2
  measurement_concept_id standard_concept_name
                   <dbl> <chr>
1                4012477 Glucose tolerance test
2                3004410 Hemoglobin A1c/Hemoglobin.total in Blood
3                3037110 Fasting glucose [Mass/volume] in Serum or Plasma
4                3009413 Insulin [Units/volume] in Serum or Plasma --fasting
5                4229586 Glucose measurement, 2 hour post prandial
6                3038553 Body mass index (BMI) [Ratio]
7                3017250 Creatinine [Mass/volume] in Urine
```

```
unique(dataset_14569019_measurement_df$unit_source_value) # lab test result
units
```

```
 [1] "No matching concept" "%"                   "415067009"
 [4] "3314211000001106"    "[{of'total'H}]"      "118582008"
 [7] "%{of'HGB}"           "s/{CO}"              NA
[10] "g/dL"                "%{tot'prot}"         "mg/dL"
[13] "10*-6.[iU]/mL"       "258896009"           "[lk_us]"
[16] "kg/m2"               "{ratio}"             "[U]"
[19] "."                   "mg/mL"               "258797006"
[22] "mg"                  "g/L"
```

```
# person data set
unique(dataset_14569019_person_df[, c("gender_concept_id", "gender")]) #
gender
```

```
# A tibble: 7 × 2
  gender_concept_id gender
              <dbl> <chr>
1            903096 PMI: Skip
2          45878463 Female
3          45880669 Male
4           1585841 Gender Identity: Non Binary
```

```
5           1585842 Gender Identity: Transgender
6           1177221 I prefer not to answer
7        2000000002 Not man only, not woman only, prefer not to answer, or
skip…
```

```
unique(dataset_14569019_person_df[, c("race_concept_id", "race")]) # race
```

```
# A tibble: 10 × 2
   race_concept_id race
             <dbl> <chr>
 1        45882607 None of these
 2            8516 Black or African American
 3      2000000008 More than one population
 4            8657 American Indian or Alaska Native
 5            8527 White
 6            8515 Asian
 7          903096 PMI: Skip
 8      2100000001 None Indicated
 9         1177221 I prefer not to answer
10        38003615 Middle Eastern or North African
```

```
unique(dataset_14569019_person_df[, c("ethnicity_concept_id", "ethnicity")])
# ethnicity
```

```
# A tibble: 5 × 2
  ethnicity_concept_id ethnicity
                 <dbl> <chr>
1              1586148 What Race Ethnicity: Race Ethnicity None Of These
2             38003563 Hispanic or Latino
3             38003564 Not Hispanic or Latino
4               903096 PMI: Skip
5               903079 PMI: Prefer Not To Answer
```

```
unique(dataset_14569019_person_df[, c("sex_at_birth_concept_id",
"sex_at_birth")]) # sex
```

```
# A tibble: 5 × 2
  sex_at_birth_concept_id sex_at_birth
                    <dbl> <chr>
1                45878463 Female
2                45880669 Male
3                       0 No matching concept
4                  903096 PMI: Skip
5                 1177221 I prefer not to answer
```

```
# survey data set
unique(dataset_14569019_survey_df[, c("answer_concept_id", "answer")]) #
question and answer
```

```
# A tibble: 24 × 2
   answer_concept_id answer
               <dbl> <chr>
```

```
 1          43529932 Including yourself, who in your family has had type 2
diab…
 2          43529668 Including yourself, who in your family has had
other/unkno…
 3            903096 PMI: Skip
 4          43529937 Including yourself, who in your family has had type 2
diab…
 5          43529938 Including yourself, who in your family has had type 2
diab…
 6           1585946 Highest Grade: College One to Three
 7           1585945 Highest Grade: Twelve Or GED
 8           1585947 Highest Grade: College Graduate
 9          43529935 Including yourself, who in your family has had type 2
diab…
10           1585948 Highest Grade: Advanced Degree
# ℹ 14 more rows
```

Selecting Appropriate Variables

```
# create new data sets with only variables of interest and renaming them
accordingly
# conditions data set
df_allofus_condition <- dataset_14569019_condition_df %>%
  select(person_id,
         diagnosis = condition_concept_id, # TB or T2D diagnosis
         diagnosis_date = condition_start_datetime,
         diagnosis_location = visit_occurrence_concept_name) # visit type
when diagnosed

# measurement data set
df_allofus_measurement <- dataset_14569019_measurement_df %>%
  select(person_id,
         test_id = measurement_concept_id,
         test_date = measurement_datetime,
         test_result = value_as_number,
         test_result_units = unit_source_value)

# person data set
df_allofus_person <- dataset_14569019_person_df %>%
  select(person_id,
         sex = sex_at_birth_concept_id,
         date_of_birth,
         race = race_concept_id,
         ethnicity = ethnicity_concept_id)

# survey data set
df_allofus_survey <- dataset_14569019_survey_df %>%
  select(person_id,
```

```
          question,
          answer_concept_id)
```

Cleaning Conditions Data set

The conditions data set is structured such that each visit has its own entry. To clean the data set, the earliest inpatient visit time at which the individual was diagnosed with TB and T2D needs to be identified. Then, the data needs to be organized into a wider table with the dates of TB and T2D diagnosis and the location.

```
# table which specifies which codes correspond to TB, T2D, and other (other
is mainliy diagnosis which is complications resulting from TB or T2D)

lookup_diagnosis <- data.frame(
  diagnosis= c(45757474, 201826, 4099651, 4193704, 435734, 4087419, 46272568,
4056331, 260936, 45763768, 434559, 255175, 432541, 253954, 45771090, 261774,
375732, 139722, 434557, 201779, 198674, 438673, 256896, 374270, 197204,
73686, 4200662, 78672, 196019, 134851, 193955, 73122, 260315, 260630, 253121,
255454, 256018, 192818, 441775, 432236, 261495, 4068849, 4040447, 4153291,
72817, 81496, 140018, 4098475, 257775, 4006347, 4224176, 258335, 4150516,
194268, 254266, 252235, 4196141, 37016768, 4221495, 43531578, 4338897,
442793, 43531616, 4008576, 4009303, 4194404, 443733, 376065, 43530690,
4222876, 37016349, 444369, 4032787, 45757363, 4228443, 45770830, 380097,
45757435, 45770881, 4222415, 43531563, 43530656, 4131908, 443729, 37017432,
43530685, 443731, 4056128, 4290822, 4266637, 45773064),
  diagnosis_recode= c(rep(2, 4), rep(1, 52), rep(3, 35)) # 1 = TB, 2 = T2D, 3
= Other
)

# Re-coding variables such that they match the specification in the look-up
table for diagnosis to be categorized as TB, T2D, or Other. If there is a
value that does not match the look-up table, then it is retained. Lastly, if
there is a missing or no value, gets coded as NA

recode_allofus_condition <- df_allofus_condition %>%
  left_join(lookup_diagnosis, by = "diagnosis") %>%
  mutate(
    diagnosis = ifelse(!is.na(diagnosis_recode), diagnosis_recode, NA)
  ) %>%
  select(-diagnosis_recode)

# The conditions are filtered by visit type to identify only the outpatient
visit locations. Then, the data is compressed such that the earliest time for
T2D diagnosis and TB diagnosis are retained in their own columns. There will
only be one column for each person_id.

# specifying the in-patient locations to keep
locations_to_keep <- c("Office Visit", "Home Visit", "Telehealth",
```

```
"Outpatient Hospital", "Outpatient Visit")

final_allofus_conditions <- recode_allofus_condition %>%
  filter((diagnosis_location %in% locations_to_keep |
is.na(diagnosis_location)) &
    diagnosis %in% c(1, 2)) %>% # keeping the visits with missing location
information
  group_by(person_id, diagnosis) %>%
  slice_min(diagnosis_date, with_ties = FALSE) %>%
  ungroup() %>%

  # the data set is separated so that the T2D and TB information is in its
own separate columns, allowing for the data to be compressed such that each
person_is is in one row
  pivot_wider(
    id_cols = person_id,
    names_from = diagnosis,
    values_from = c(diagnosis_date, diagnosis_location),
    names_sep = "_"   # creates unique names like diagnosis_date_1,
diagnosis_location_1
  ) %>%
  rename(
    tb_date = diagnosis_date_1,
    tb_diagloc = diagnosis_location_1,
    t2d_date = diagnosis_date_2,
    t2d_diagloc = diagnosis_location_2
  )
```

Cleaning Labs/Measurements Dataset

The measurement data set is structured similarly to the conditions, with each visit having its own entry. To clean the data set, the labs with the closest result to the time of diagnosis are retained.

```
# The different tests in the variable test date are first identified then T2D
diagnosis time is joined to the data for comparison. The absolute value
difference is used to identify and keep the test closest to the time of
diagnosis.

final_allofus_measurements <- df_allofus_measurement %>%
    mutate(test_date = as.POSIXct(test_date), # changing date time from
character string to time
    test_id = case_when(
      test_id == 3009413 ~ "insulin_fasting",
      test_id == 3037110 ~ "glucose_fasting",
      test_id == 4012477 ~ "pogtt",
      test_id == 4229586 ~ "post_prandial_glucose",
      test_id == 3004410 ~ "hba1c",
      test_id == 3017250 ~ "creatinine",
```

```
      test_id == 3038553 ~ "bmi",
      TRUE ~ NA_character_
    )
  ) %>%
  left_join(final_allofus_conditions %>%
              mutate(t2d_date = as.POSIXct(t2d_date)) %>% # changing date
time from character string to time
              select(person_id, t2d_date),
            by = "person_id") %>%
  filter(!is.na(t2d_date)) %>%
  mutate(days_from_t2d = abs(as.numeric(difftime(test_date, t2d_date, units =
"days")))) %>%
  group_by(person_id, test_id) %>%
  slice_min(days_from_t2d, with_ties = FALSE) %>%
  ungroup() %>% # selecting the columns/variable to keep for final data set
  pivot_wider(
    id_cols = person_id,
    names_from = test_id,
    values_from = c(test_result, test_result_units),
    names_glue = "{test_id}_{.value}"
  )
```

Recoding Person Dataset

```
# The person data set is re-coded to be similar to the other data sets
final_allofus_person <- df_allofus_person %>%
  mutate(

    # sex
    sex = case_when(
      sex == 45880669   ~ 1, # Male
      sex == 45878463   ~ 2, #Female,
      sex %in% c(0, 903096, 1177221) ~ 9, # Skip or no answer
      TRUE ~ NA_real_
      ),

    # race
    race = case_when(
      race == 8516 ~ 0, # Black
      race == 8527 ~ 1, #   White
      race %in% c(45882607, 8657, 2000000008, 8515, 38003615)  ~ 3, #
Other/multicultural
      race %in% c(903096, 1177221, 2100000001) ~ 9, # Skip or no answer
      TRUE ~ NA_real_
      ),

    # ethnicity
    ethnicity = case_when(
```

```
      ethnicity == 38003563 ~ 1, #  Hispanic or Latino
      ethnicity == 38003564 ~ 0, # Not Hispanic or Latino
      ethnicity %in% c(1586148, 903096, 903079) ~ 9, # Skip or no answer
      TRUE ~ NA_real_
    ),

    # DOB from character to date
    date_of_birth = as.POSIXct(date_of_birth)
  )
```

Re-coding and expanding the survey table to create appropraite variable

The survey data set had all the different survey questions and answer (ex: Highest Grade: College Graduate) under one variable name called answer. To form the various variables, the data set is filtered by the question then re-coded to be similar to the other data sets.

```
# The survey data is also re-coded similar to be similar to the other data
sets.
# make education variable
final_education <- df_allofus_survey %>%
  filter(question == "Education Level: Highest Grade") %>%
  mutate(
    education = case_when(
      answer_concept_id == 1585948 ~ 4,                  # graduate studies
      answer_concept_id %in% c(1585946, 1585947) ~ 3,    # college
      answer_concept_id %in% c(1585945, 1585944) ~ 2,    # high school
graduate
      answer_concept_id %in% c(1585943, 1585942) ~ 1,    # less than high
school
      answer_concept_id %in% c(903096, 1585941, 903079) ~ 9,  # did not
answer / No school
      TRUE ~ NA_real_
    )
  ) %>%
  select(person_id, education)

# make family history variable
library(dplyr)

final_family_hx <- df_allofus_survey %>%
  filter(question %in% c(
    "Including yourself, who in your family has had type 2 diabetes? Select
all that apply.",
    "Including yourself, who in your family has had other/unknown diabetes?
Select all that apply."
  )) %>%
  mutate(
    family_hx_db = case_when(
```

```
      answer_concept_id %in% c(
        43529669, 43529671, 43529672, 43529673, 43529674, 43529675,
        43529933, 43529935, 43529936, 43529937, 43529938, 43529939
      ) ~ 1,  # yes
      answer_concept_id %in% c(903096, 903079) ~ 9, # did not answer
      TRUE ~ NA_real_
    )
  ) %>%
  select(person_id, family_hx_db) %>%
  group_by(person_id) %>%
  summarise(
    family_hx_db = case_when(
      any(family_hx_db == 1, na.rm = TRUE) ~ 1,
      any(family_hx_db == 9, na.rm = TRUE) ~ 9,
      TRUE ~ NA_real_
    ),
    .groups = "drop"
  )
```

Merging all the All of Us Datasets Together

```
# Merging all the data sets together for final all of us
allofus_cleaned <- final_allofus_conditions %>%
  left_join(final_allofus_measurements, by = "person_id") %>%
  left_join(final_allofus_person, by = "person_id") %>%
  left_join(final_education, by = "person_id") %>%
  left_join(final_family_hx, by = "person_id")
```

Labelling the Dataset

```
# Labeling the data set to be similar to the other data sets
label_allofus <- function(allofus_cleaned) { allofus_cleaned %>%
    mutate(
      # Sex
      sex = factor(sex,
                   levels = c(1, 2),
                   labels = c("Men", "Women")),

      # Family history
      family_hx_db = factor(family_hx_db, levels = c(1, 9),
                            labels = c("Yes", "Missing")),

      # Race/Ethnicity
      race = factor(race, levels = c(0, 1, 2, 3, 9),
                    labels = c("Black", "White",
                         "Mexican/Other Hispanic",
                         "Other/Multiracial",
                         "Missing")),
```

```
      ethnicity = factor(ethnicity,
                         levels = c(0, 1, 9),
                         labels = c("Non-Hispanic", "Hispanic", "Missing")),

      # Education
      education = factor(education,
                         levels = c(1, 2, 3, 4, 9),
                         labels = c("< High School", "High School Grad",
                              "Some College/AA",
                              "College+",
                              "Refused/DK"))
    )
}

allofus_final <- label_allofus(allofus_cleaned)
```

Exploratory Data Analysis

```
library(dplyr)
library(gtsummary)

vars_to_summarize <- c(
  "sex", "race", "ethnicity", "education",
  "family_hx_db", "glucose_fasting_test_result", "hba1c_test_result"
)

tbl_gts <- allofus_final %>%
  select(all_of(vars_to_summarize)) %>%
  tbl_summary(
    statistic = list(
      all_continuous() ~ "{mean} ({sd})",
      all_categorical() ~ "{n} / {N} ({p}%)"
    ),
    digits = list(
      all_continuous() ~ 2,
      all_categorical() ~ c(0, 0, 1)
    ),
    label = list(
      sex ~ "Sex",
      race ~ "Race/Ethnicity",
      ethnicity ~ "Ethnicity",
      education ~ "Education",
      family_hx_db ~ "Family history of DM",
      glucose_fasting_test_result ~ "Fasting Glucose (mg/dl)",
      hba1c_test_result ~ "HbA1c (%)"
    ),
    missing = "no"
```

```
  ) %>%
  modify_header(
    label ~ "**Variables**",
    stat_0 ~ "**Summary Statistics (All of Us)**"
  ) %>%
  bold_labels() %>%
  italicize_levels()

tbl_gts
```

Variables	Summary Statistics (All of Us)[1]
Sex	
Men	223 / 584 (38.2%)
Women	361 / 584 (61.8%)
Race/Ethnicity	
Black	174 / 598 (29.1%)
White	127 / 598 (21.2%)
Mexican/Other Hispanic	0 / 598 (0.0%)
Other/Multiracial	68 / 598 (11.4%)
Missing	229 / 598 (38.3%)
Ethnicity	
Non-Hispanic	352 / 598 (58.9%)
Hispanic	232 / 598 (38.8%)
Missing	14 / 598 (2.3%)
Education	
< High School	75 / 598 (12.5%)
High School Grad	189 / 598 (31.6%)
Some College/AA	242 / 598 (40.5%)
College+	62 / 598 (10.4%)
Refused/DK	30 / 598 (5.0%)
Family history of DM	
Yes	85 / 150 (56.7%)
Missing	65 / 150 (43.3%)
Fasting Glucose (mg/dl)	115.97 (38.78)
HbA1c (%)	23,536.59 (485,070.90)

[1] n / N (%); Mean (SD)

Separating the Dataset by Diagnosis Time

The data set is separated such that one data set contains participants who had TB first, then T2D, and another data set contains participants who had T2D first, then TB.

```
# Separating the data set by diagnosis time
#tb_before_t2d  <- allofus_final %>% filter(tb_date < t2d_date)
```

```
#t2d_before_tb  <- allofus_final %>% filter(t2d_date < tb_date)
#t2d_tb_equal <- allofus_final %>% filter(t2d_date == tb_date)

# making it into a separate variable instead
#allofus_final <- allofus_final %>%
#  mutate(tb_t2d_relation = case_when(
#    tb_date < t2d_date  ~ "tb_before_t2d",
#    t2d_date < tb_date  ~ "t2d_before_tb",
#   tb_date == t2d_date ~ "tb_t2d_same_date",
#    TRUE                ~ NA_character_
#   ))
```